JAVA MULTITHREADING

A **thread** = a lightweight unit of execution. Same heap, own stack.

2 Ways to Create a Thread

① extend Thread

```
class MyThread extends Thread {
  public void run() {
    print("working...");
  }
}
new MyThread().start();
```

✗ burns your only inheritance slot

② implement Runnable

preferred ✓

```
class MyTask implements Runnable {
  public void run() { ... }
}
new Thread(new MyTask()).start();
// java 8+
new Thread(() -> work()).start();
```

✓ splits the task from the worker

→ that same Runnable is what you later hand to an ExecutorService / thread pool

Essential Methods

`start()` — spawns a NEW thread, calls run()

`run()` — the work. NO new thread

`sleep(ms)` — pause the current thread

`join()` — wait for a thread to finish

`interrupt()` — wake a sleeping / waiting thread

`isAlive()` — started & not done yet?

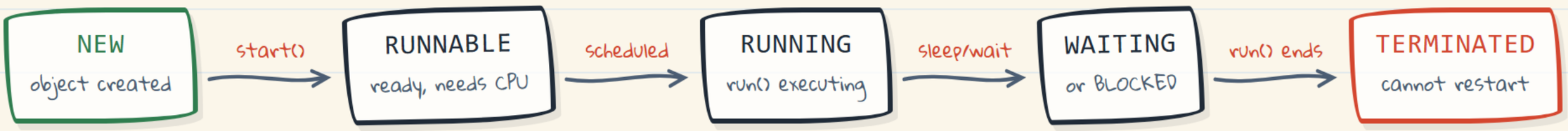
`currentThread()` — who runs this code?

`setName()` — readable thread dumps

`yield()` — hint: "others can go first"

`setPriority()` — 1–10, only a hint

Where a Thread Lives



⚠ the interview trap

`t.run()` → runs on the main thread

`t.start()` → runs on a new thread

up next →

synchronized, wait / notify, then why `new Thread()` is rarely what you want in production.

#Java #Multithreading #LearningInPublic

synchronized

*One thread at a time on that **monitor**. Everyone else waits at the door.*

2 Ways to Lock

① on a method

```
public synchronized void add() {
  count++;   // lock is "this"
}

public static synchronized void inc() {
  TOTAL++;  // lock is the Class
}
```

whole method is the critical section

② on a block *tighter*

```
public void add() {
  // lock a private object
  synchronized (lock) {
    count++;
  }
}
```

✓ lock only what you must. prefer a *private lock*

Rules I Keep Repeating

`monitor` — every Java object has one

`auto-release` — unlocks even if you throw

`not fair` — no queue. no tryLock()

`reentrant` — same thread can lock again

`visibility` — unlock → next lock sees the writes

`same lock` — different objects = not synced

What You Are Actually Locking

instance method
lock = this

static method
lock = MyClass.class

block
lock = the object you pass

⚠ the interview trap

locking two different objects does nothing
never `synchronized (this)` on a public API

up next →

wait / notify — they only work if you already hold the monitor.

#Java #Multithreading #LearningInPublic

wait / notify / notifyAll

These live on **Object**, not Thread. Sleep pauses. Wait **hands the lock back**.

The Three Calls

`wait()` — release the lock, sleep until notified

`notifyAll()` — wake ALL waiters — safer default

`notify()` — wake ONE waiter on this monitor

`no lock` — IllegalMonitorStateException

wait() vs sleep()

sleep(ms)

```
Thread.sleep(1000);
// still HOLDS the lock
// wakes by itself after time
```

✗ other threads stay blocked on you

wait()

```
synchronized (lock) {
  while (!ready) lock.wait();
}
// RELEASES the lock first
```

✓ someone else can enter and notify

Producer / Consumer Shape

```
synchronized (queue) {
  while (queue.isEmpty()) queue.wait();  // always while, never if
  item = queue.remove();
}
synchronized (queue) {
  queue.add(item);
  queue.notifyAll();                     // wake consumers
}
```

→ always wait inside a while loop — notify can be a false alarm (spurious wakeup)

⚠ the interview trap

`wait()` without `synchronized` → crash

prefer `notifyAll()` until you can prove notify is safe

up next →

volatile vs Atomic vs synchronized. Three tools. Three different jobs.

#Java #Multithreading #LearningInPublic

volatile · Atomic · synchronized

Pick the tool by the job: **see it**, **count it**, or **guard a whole story**.

Three Jobs

volatile

```
volatile boolean stop;

// writer
stop = true;

// reader always sees it
while (!stop) work();
```

visibility + ordering.
✗ not atomic. count++ still races.

Atomic*

```
AtomicInteger n
  = new AtomicInteger();

n.incrementAndGet();
n.compareAndSet(old, neu);
```

CAS on one variable. no lock.
great for counters & flags.

synchronized

```
synchronized (lock) {
  if (balance >= amt)
    balance -= amt;
}
```

mutual exclusion for a compound action. check-then-act lives here.

Say This Out Loud

"volatile gives visibility and ordering — not mutual exclusion, and not atomicity of count++."

that sentence is half the interview.

When I Reach For What

`stop flag` — volatile boolean

`check then update` — synchronized or a lock

`hit counter` — AtomicInteger / LongAdder

`publish config` — volatile ref to an immutable object

⚠ the interview trap

`volatile int count; count++;`

read + add + write. three steps. still a race.

up next →

why `new Thread()` is rarely what you want in production.

#Java #Multithreading #LearningInPublic

WHY NOT new Thread()

Threads are expensive. A pool of reusable workers + a **bounded queue** is the grown-up API.

new Thread() vs ExecutorService

new Thread(task).start()

- ✗ unbounded: one request = one OS thread
- ✗ no queue, no backpressure
- ✗ you manage lifecycle yourself
- ✗ unnamed → garbage thread dumps

ExecutorService

- ✓ reuse a fixed set of workers
- ✓ queue extra work, reject when full
- ✓ shutdown / awaitTermination
- ✓ name the threads. future-you thanks you

The Shape I Actually Write

```
ExecutorService pool = new ThreadPoolExecutor(
    4, 16,                      // core, max
    60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(200),
    namedFactory("http-%d"),
    new ThreadPoolExecutor.CallerRunsPolicy()
);
```

execute vs submit

`execute(r)` — exceptions go to the thread handler (often lost)

`submit(r)` — returns Future. you must get() or they vanish

`shutdown()` — no new tasks, finish the rest

`shutdownNow()` — interrupt workers, return leftovers

⚠ the interview trap

`Executors.newFixedThreadPool(n)` uses an **unbounded** queue — pool never grows, heap can explode

up next →

deadlock, livelock, starvation. how they look, how you find them.

#Java #Multithreading #LearningInPublic

DEADLOCK & FRIENDS

Two threads, two locks, opposite order. Nobody moves. Coffee gets cold.

The Classic Picture

Thread A

```
synchronized (lock1) {
  synchronized (lock2) {
    work();
  }
}
```

holds 1, wants 2

Thread B

```
synchronized (lock2) {
  synchronized (lock1) {
    work();
  }
}
```

holds 2, wants 1 → stuck forever

Three Different "Stuck"

deadlock

circular wait. zero progress. dump shows BLOCKED.

livelock

everyone is busy yielding / retrying. still no progress.

starvation

one thread never gets the lock. unfair + hot threads.

How I Talk About Fixes

`lock order` — always 1 then 2. never the reverse

`fewer locks` — or immutable data. no share, no deadlock

`tryLock` — timeout + backoff, then retry

`jstack / jcmd` — "Found one Java-level deadlock"

⚠ the interview trap

deadlock is silent. CPU looks idle. take a thread dump, don't guess.

series done →

next notebook: ThreadLocal, CompletableFuture, virtual threads.

#Java #Multithreading #LearningInPublic